Proper Scoring Rules and Calibration

ELG 5218 - Uncertainty Evaluation in Engineering Measurements and Machine Learning

Miodrag Bolic

University of Ottawa

March 4, 2026

Outline: Proper Scoring Rules and Calibration

- **Motivation & Definitions**
- **Proper Scoring Rules:** NLL / Brier / CRPS
- **Calibration Diagnostics:** reliability diagram, ECE
- **Empirical Findings:** modern deep nets are miscalibrated
- **Calibration Methods:** temperature scaling, Platt, histogram, isotonic
- **Applications & Notebooks:** image classification, detection, LLMs
- **Case Study & Code Demo**
- **Wrap-up & Reading/Assignments**

Next lecture: conformal prediction (finite-sample coverage, efficiency).

Why do we care about *probabilities*?

- Many decisions need **risk-aware** predictions (medicine, autonomy, finance).
- **Accuracy is not enough:** two models with identical accuracy can have very different uncertainty quality.
- We want a metric that rewards:
  - **Calibration:** predicted probabilities match empirical frequencies.
  - **Sharpness:** confident when correct, uncertain when unsure.

Key question: how do we score a *predictive distribution* $p_\theta(y \mid x)$?

Scoring rules (setup)

We observe (x, y) and a model outputs a predictive distribution $p_\theta(y \mid x)$.

A **scoring rule** assigns a number $S(p_\theta, (x, y))$.

- Higher score (or lower loss) means better probabilistic prediction.
- We care about the **expected** score under the true data-generating process.

Proper scoring rule: the expected score is optimized when the predictive distribution equals the true conditional distribution:

$$\mathbb{E}_{(x,y)\sim p^*}\left[S(p_\theta, (x, y))\right] \leq \mathbb{E}_{(x,y)\sim p^*}\left[S(p^*, (x, y))\right]$$

(with equality iff $p_\theta(y \mid x) = p^*(y \mid x)$). In practice, we approximate this expectation by an average over a test set.

Reference: Murphy, *Probabilistic Machine Learning: Advanced Topics*, Sec. 14.2.1.

Binary intuition: why "proper" matters

Binary event $y \in \{0, 1\}$. Suppose the true probability is $q = p^*(y = 1)$, and the model reports p. Here q is the (unknown) true event probability, and p is the forecaster's reported probability $p = p_\theta(y = 1)$.

Log score (NLL):

$$\ell_{\text{NLL}}(p; y) = -\left[y \log p + (1 - y) \log(1 - p)\right]$$

Then the expected loss is

$$\mathbb{E}[\ell_{\text{NLL}}(p; y)] = -\left[q \log p + (1 - q) \log(1 - p)\right]$$

Taking derivative and setting to zero gives optimum at $p = q$ (strictly proper).

Key takeaway: proper scores discourage "hedging" or systematically over/under-confident probabilities.

NLL / Log loss (classification)

Negative log-likelihood (NLL) is the standard proper scoring rule for probabilistic classifiers:

$$\ell_{\text{NLL}} = -\log p_\theta(y \mid x)$$

- Equivalent to cross-entropy for one-hot labels.
- Strongly penalizes assigning tiny probability to the true label ("tail sensitive").
- Minimizing NLL is aligned with maximizing likelihood / minimizing KL to the true distribution.

In Murphy's text, the log-likelihood is presented as a proper scoring rule and linked to Gibbs' inequality.

Brier score (classification)

For C-class classification with probabilities $\hat{p}_c = p_\theta(y = c \mid x)$ and one-hot label $\mathbb{I}(y = c)$,

$$\text{Brier} = \frac{1}{C} \sum_{c=1}^{C} (\hat{p}_c - \mathbb{I}(y = c))^2$$

- Proper scoring rule.
- Less sensitive than NLL to extreme probabilities.
- Interpretable as squared error between predicted probability vector and one-hot label.

Original: Brier (1950). Modern ML discussion in Murphy, Sec. 14.2.1.

CRPS (regression / continuous outcomes)

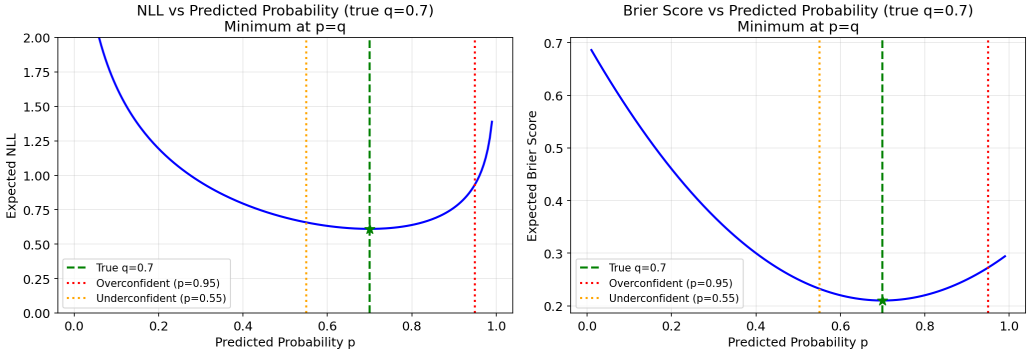
For continuous y and predictive CDF F, the **continuous ranked probability score (CRPS)** is:

$$\mathrm{CRPS}(F, y) = \int_{-\infty}^{\infty} \left(F(z) - \mathbb{I}(z \geq y)\right)^2 dz$$

- Strictly proper scoring rule for distributions on $\mathbb{R}$.
- Generalizes absolute error from points to distributions.
- Useful when we predict full distributions (e.g., Gaussian mean+variance) or quantiles.

Theory: Gneiting & Raftery (JASA, 2007). Practical implementations exist in many libraries.

Visual: NLL vs Brier penalties (binary)



Interpretation: With true $q = 0.7$ (green), both expected NLL and expected Brier achieve their minimum at $p = q$. Moving to an underconfident forecast $p = 0.55$ (orange) increases both losses moderately, while an overconfident forecast $p = 0.95$ (red) increases expected NLL much more.

Calibration: definition

A probabilistic classifier is **calibrated** if:

$$p^*(y = 1 \mid \hat{p} = 0.9) \approx 0.9$$

More generally, for multiclass, the predicted confidence should match empirical accuracy.

- Calibration is about **probabilities**, not just class predictions.
- Proper scoring rules (like NLL) encourage calibration in theory, but modern deep nets can still be miscalibrated.

Murphy, Sec. 14.2.2.

Reliability diagram + ECE (Expected Calibration Error)

Reliability diagram: bin predictions by confidence; plot (bin accuracy) vs (bin confidence).

With B bins, let B_b be indices whose confidence falls in bin b.

$$\mathrm{acc}(B_b) = \frac{1}{|B_b|} \sum_{i \in B_b} \mathbb{I}(\hat{y}_i = y_i), \qquad \mathrm{conf}(B_b) = \frac{1}{|B_b|} \sum_{i \in B_b} \hat{p}_i$$

ECE:

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{N} \left|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)\right|$$

ECE and reliability diagrams summarized in Murphy, Sec. 14.2.2.1.

Notes on ECE

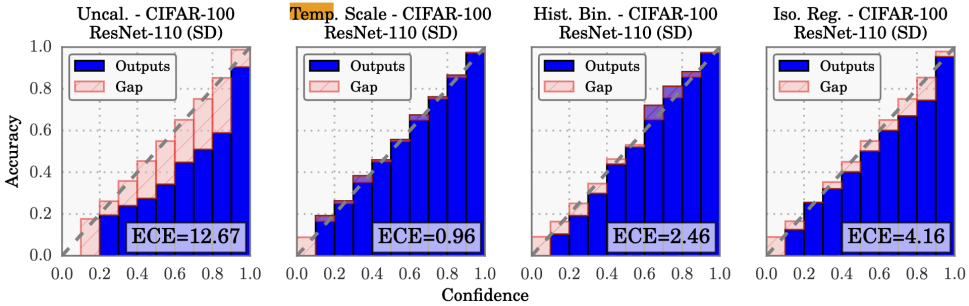
- ECE depends on **binning** choices (uniform bins vs equal-frequency bins).
- ECE often uses only the **top-1 confidence** in multiclass settings; this can miss miscalibration of non-top classes.
- Alternatives:
  - classwise / marginal calibration error,
  - adaptive binning,
  - differentiable calibration losses (for training-time objectives).

Modern deep nets: accurate but miscalibrated

Empirical result (Guo et al., ICML 2017):

- Modern architectures (ResNet, DenseNet, etc.) are often **overconfident**.
- Depth, width, weight decay, and BatchNorm can **worsen** calibration.
- A simple post-hoc fix, **temperature scaling**, is surprisingly effective.

Visual: Temperature Scaling



Source: "On Calibration of Modern Neural Networks" (ICML 2017).

Post-hoc calibration: Platt vs Temperature scaling

Let $\boldsymbol{z}$ be logits from a trained classifier (fixed model). We learn a small calibration map on a validation set.

Platt scaling (binary): $q = \sigma(az + b)$
Temperature scaling (multiclass):

$$q = \mathrm{softmax}(\boldsymbol{z}/T)$$

- Single parameter $T > 0$ fitted by minimizing NLL on a calibration set.
- Does *not* change the predicted class (argmax unchanged).
- Often strong performance and very stable.

Murphy, Sec. 14.2.2.5; Guo et al., ICML 2017.

Platt vs Temperature scaling: training vs inference

Platt scaling

Model output: score/logit $z(x)$ (frozen model)
Calibration map (binary): $q(x) = \sigma(az(x) + b)$
Post-hoc training (calibration set):
$\min_{a,b} \sum_{i \in D_{\text{cal}}} -\Big[y_i \log q_i + (1 - y_i) \log(1 - q_i)\Big]$
Optimization: convex; use LBFGS / Newton / SGD.
Inference: $\hat{q}(x) = \sigma(az(x) + b)$
Multiclass extension (common options):

- One-vs-rest: fit (a_c, b_c) per class using $z_c(x)$.
- Matrix scaling: $q = \mathrm{softmax}(Wz + b)$ (more flexible, more params).

Temperature scaling

Model output: logits $\boldsymbol{z}(x) \in \mathbb{R}^C$ (frozen)
Calibration map: $q(x) = \mathrm{softmax}\big(\boldsymbol{z}(x)/T\big), \quad T > 0$
Post-hoc training (calibration set): $\min_{T>0} \sum_{i \in D_{\text{cal}}} -\log q_i(y_i)$
Optimization: 1D; optimize $\log T$ with LBFGS / gradient descent.
Inference: $\hat{q}(x) = \mathrm{softmax}(\boldsymbol{z}(x)/T)$
Properties:

- Preserves class ranking $\Rightarrow$ argmax unchanged.
- $T > 1$: softer (less confident); $T < 1$: sharper.

Nonparametric calibration: histogram binning & isotonic regression

Histogram binning:

- Partition predicted confidences into bins.
- Replace predictions in each bin by the empirical accuracy in that bin.

Isotonic regression:

- Learn a monotone non-decreasing function $q = f(p)$ from validation data.
- More flexible than Platt scaling, but can overfit with small calibration sets.

Murphy, Sec. 14.2.2.4.

Histogram binning vs Isotonic regression: training vs inference

Histogram binning

Inputs: predicted confidence p_i and label $y_i \in \{0, 1\}$ on Dcal.
Post-hoc training:

- Choose K bins over $[0, 1]$ (equal-width or equal-count).
- For each bin B_k, compute empirical accuracy:

$$\hat{a}_k = \frac{1}{|B_k|} \sum_{i \in B_k} y_i$$

Inference:

$$\hat{q}(p) = \hat{a}_k \quad \text{if } p \in B_k$$

Notes: very simple; can be noisy if bins have few samples.

Isotonic regression

Goal: learn a monotone function $f : [0, 1] \to [0, 1]$.
Post-hoc training (calibration set):

$$\min_{f \text{ nondecreasing}} \sum_{i \in D\text{cal}} (y_i - f(p_i))^2$$

Algorithm: Pool-Adjacent-Violators (PAV) $\Rightarrow$ produces a *monotone step function*.
Inference:

$$\hat{q}(p) = f(p)$$

Notes: more flexible than Platt (not restricted to sigmoid), but can overfit with small Dcal.

Practical guidance (what to do in projects)

- Always keep a **calibration set** separate from training data.
- Report both:
  - **discrimination** (accuracy / AUROC),
  - **calibration** (reliability plot, ECE, NLL, Brier).
- Post-hoc methods are cheap and easy; temperature scaling is a strong default.
- Under distribution shift, calibration can degrade (monitor on new domains).

Applications: same idea, different outputs

Calibration tools are modular: you need **(confidence, correctness)** pairs.

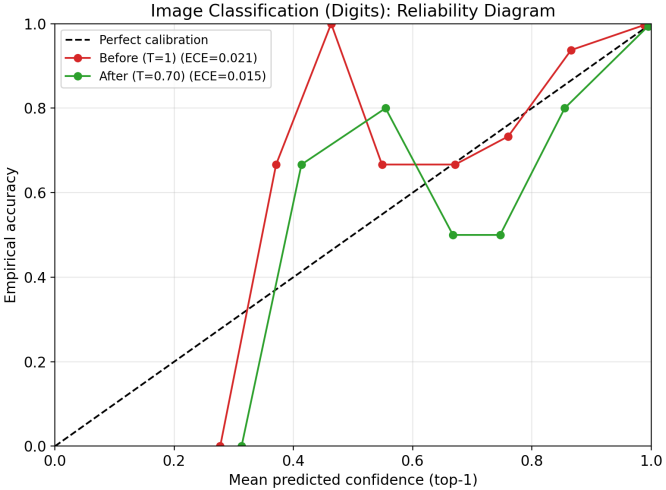
- **Image classification:** confidence = top-1 probability; correctness = (top-1 is correct).
- **Object detection:** confidence = detection score; correctness = (matched to GT with correct class + IoU$\geq \tau$).
- **LLMs/VLMs:** confidence = "probability answer is correct" (self-reported or derived); correctness = evaluation label.

Same diagnostics: reliability diagram, ECE, NLL/Brier (binary), NLL (multiclass).

Image classification example: temperature scaling (Digits)

- train a multinomial classifier on `digits`,
- fit a single temperature T on a calibration split by minimizing NLL,
- show reliability diagram + ECE before/after.

Image classification example: temperature scaling (Digits)



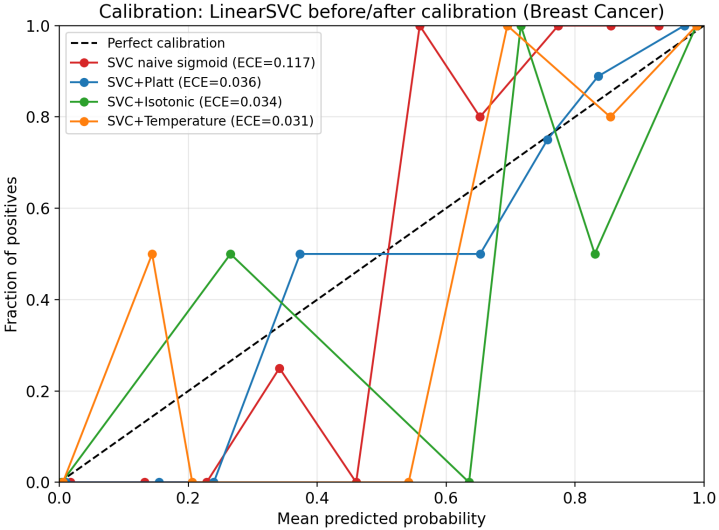
Hands-on notebooks

Two practical resources to base your in-class demo and lab on:

- scikit-learn calibration example: `plot_calibration_curve`
- CalibrationTutorial (notebooks + explanations): `luferrer/CalibrationTutorial`

Course notebook (this lecture): `calibration_lab_notebook.ipynb`

Hands-on notebooks



Object detection: what is "confidence calibration"?

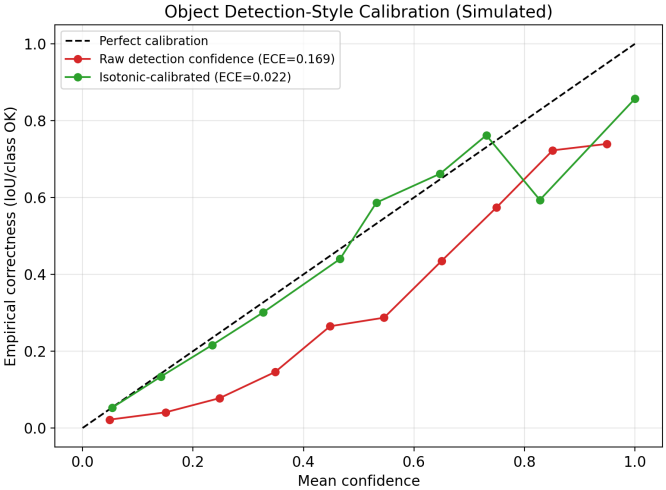
Detection confidence is often treated as **a probability of correctness**, but:

- it is not optimized for calibration (it is tuned for ranking/AP),
- it may shift across domains (weather, viewpoint, sensor, lighting),
- it may vary by class, size, occlusion, and localization quality.

A common evaluation protocol:

- label each detection as correct/incorrect using matching (class + IoU),
- plot reliability diagram of confidence vs correctness,
- calibrate using Platt/isotonic/temperature (or class-conditional variants).

Object detection: what is "confidence calibration"?



LLMs: confidence, abstention, and calibration

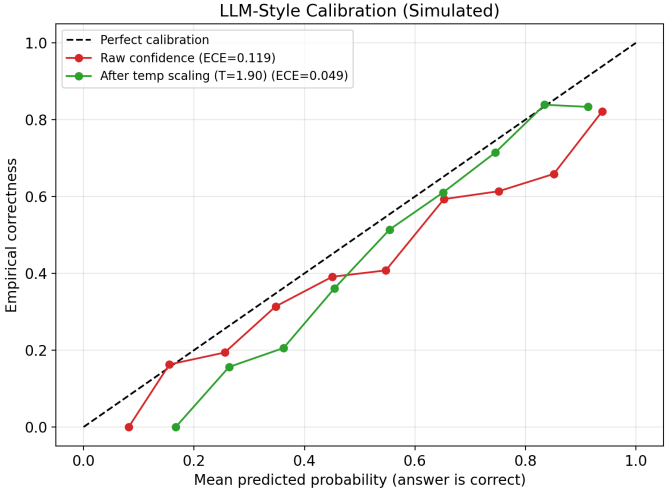
Typical setup:

- model answers a question and outputs a confidence score (explicit or derived),
- we evaluate correctness against a benchmark,
- calibrate confidence to avoid overconfident errors ("alarm fatigue" / unsafe deployment).

Practical knobs:

- **post-hoc calibration:** temperature scaling or isotonic on (confidence, correctness),
- **selective prediction:** abstain when confidence is low (ties to conformal in next lecture).

LLMs: confidence, abstention, and calibration



Assignment

Goal: reproduce + extend calibration analysis.

1. Pick one domain (image classification / detection / LLM QA).
2. Compute: NLL, Brier (if binary), reliability diagram, ECE (report binning choice).
3. Apply multiple calibration methods.
4. Report: calibration improvement *and* any change in accuracy / AP / selective risk.

Stretch: analyze calibration under a domain shift (corruptions, weather, prompt change).

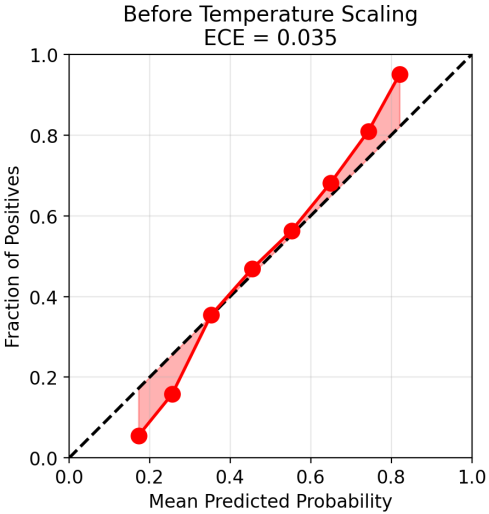
Toy demo: miscalibration + temperature scaling

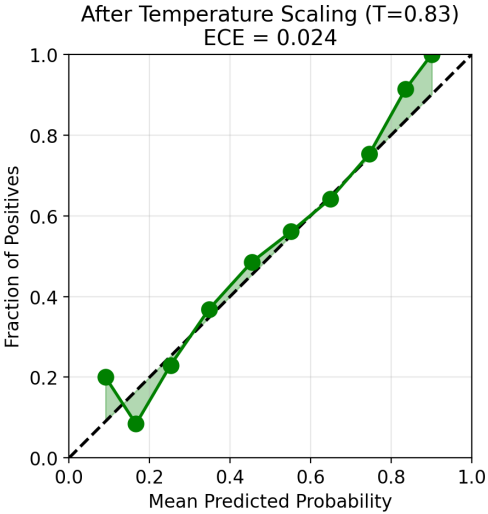
We simulate an **overconfident** classifier and then learn T on a calibration split by minimizing NLL.

- Before: high ECE (overconfident probabilities).
- After: much lower ECE and improved NLL / Brier.

The next slide shows reliability diagrams from a small Python demo (provided with this deck).

Reliability diagrams (toy example)





Toy demo metrics (test set)

Metric	Before	After (Temp. scaling)
NLL	0.603	0.598
Brier	0.208	0.206
ECE (10 bins)	0.035	0.024

In this run, learned temperature $T \approx 0.83$ (higher T softens overconfident logits).

Wrap-up: key takeaways

- Use **proper scoring rules** (NLL, Brier, CRPS) to evaluate *probabilistic* predictions.
- Calibration is about **matching probabilities to frequencies**.
- Diagnose with **reliability diagrams** and summary metrics (ECE + NLL/Brier).
- Practical default: **temperature scaling** is simple and often very effective.

Next lecture: conformal prediction (finite-sample coverage + efficiency trade-offs).

References

- Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. (Calibration + proper scoring rules, Sec. 14.2).
- Chuan Guo, Geoff Pleiss, Yu Sun, Kilian Q. Weinberger. *On Calibration of Modern Neural Networks*. ICML 2017.
- Tilmann Gneiting, Adrian E. Raftery. *Strictly Proper Scoring Rules, Prediction, and Estimation*. JASA 2007.
- Glenn W. Brier. *Verification of Forecasts Expressed in Terms of Probability*. Monthly Weather Review 1950.
- J. Oberdiek et al. *Estimating and Evaluating Regression Predictive Uncertainty in Deep Object Detectors*. arXiv:2101.05036.
- L. Küppers et al. *Multivariate Confidence Calibration for Object Detection*. CVPR Workshops 2020.
- (LLMs) Example reading: *Language Models (Mostly) Know What They Know* (confidence estimation for LMs).
- Practical notebooks: scikit-learn calibration example; *CalibrationTutorial* by L. Ferrer.